

SILVA: Parallel Spatial Index Library with Versioned Access (technical report)

Bo Huang
UC Riverside
bo.huang@email.ucr.edu

Ahmed Eldawy
UC Riverside
eldawy@ucr.edu

Yan Gu
UC Riverside
ygu@cs.ucr.edu

Yihan Sun
UC Riverside
yihans@cs.ucr.edu

Abstract

There has been a rapid rise of agentic AI systems where autonomous agents can independently query, analyze, and modify data. In spatial settings, each agent operates on a specific region, issuing numerous concurrent queries and making speculative updates that must be validated before commit. Ensuring reliability and reproducibility requires versioned access that can efficiently isolate, audit, merge, or roll back each agent's changes. This paper presents SILVA, a library of main-memory spatial indexes designed for versioned access to highly dynamic spatial data. We formalize the problem of multi-version spatial data management and propose multiple indexes that build on state-of-the-art functional data structures to support immutable versions with atomic batch updates. SILVA is highly parallel and space-efficient, enabling seamless spatial queries across versions. Experiments on synthetic and real-world datasets show that SILVA outperforms state-of-the-art indexes by up to two orders of magnitude, peaking at 112 million updates per second, while reducing memory usage by 70%.

CCS Concepts

• Information systems → Parallel and distributed DBMSs.

Keywords

Versioned Access, Spatial Data, Functional Data Structures

ACM Reference Format:

Bo Huang, Ahmed Eldawy, Yan Gu, and Yihan Sun. 2026. SILVA: Parallel Spatial Index Library with Versioned Access (technical report). In *Proceedings of The 34th ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL '26)*. ACM, New York, NY, USA, 16 pages.

Yihan: I'm considering renaming MVZD-tree as something like mvquad-tree, or mvq-tree. I think it's more essential to name it based on quad tree instead of zd-tree.

1 Introduction

Bo: technical report test.

With the rapid advancement of geospatial technologies, spatial data are generated and collected at an ever-growing scale, and are updated and shared more frequently than ever (e.g., daily crime re-

ports [20], traffic accident records [45], and demographic shifts [21]). As a result, modern applications such as collaborative mapping and agentic systems increasingly rely on analyzing how spatial data evolve over time. For these use cases, querying the "here and now" no longer suffices; instead, these systems demand the capability to traverse data timelines or even explore a hypothetical "multiverse." Examples include retrieving historical states, simulating "what-if" scenarios by editing and analyzing versions within temporary sandboxes, and comparing or merging divergent versions. In response to this broad need, many systems have been developed, including GeoGig [31], Kart [37], and others [39, 52, 68].

Nevertheless, the performance of these systems remains suboptimal due to a lack of support for parallelism; yet, enabling parallelism is highly challenging due to the intricacies of version access. One traditional approach to versioning is the spatio-temporal indexes [39, 52, 68], which casts temporal evolution as an additional time dimension. This solution, however, only supports updates to the tip of the timeline, thereby restricting it to linear versioning. Arbitrary version evolution is considerably more complex. Under Git-style version-control semantics (see Fig. 1), versions form a DAG, making it difficult to efficiently manage, modify, and query disparate versions, let alone in parallel. To our knowledge, no existing solution incorporates effective parallel updates, either for traditional spatio-temporal indices [39, 52, 68] or for Git-style frameworks such as GeoGig [31] and Kart [37]. As single-core CPU performance has largely plateaued since 2005, sequential solutions are increasingly unlikely to scale with the rapid growth of geospatial data. The closest parallel mechanism we know of, Multi-Version Concurrency Control (MVCC) [24, 40, 42, 44, 49, 53, 60, 64, 68, 71], supports concurrent updates but serves a fundamentally different purpose: it focuses on creating new versions atop the latest one upon updates, so that unfinished queries see a consistent version, rather than supporting expressive cross-version analysis or branching updates.

In this paper, we address this gap by introducing a **highly efficient, parallel, in-memory solution for managing spatial data with Git-style versioning**. Our system, SILVA¹, facilitates parallel updates and high-performance queries across arbitrary historical versions. By leveraging effective *parallelism*, SILVA **outperforms all existing baselines by at least two orders of magnitude for most update operations on a 96-core machine**. Furthermore, it demonstrates significant speedups across various query types supported by state-of-the-art systems.

Our work is motivated by recent designs of in-memory parallel functional (immutable) binary search trees (BSTs) [22, 23, 65–67], which have demonstrated effectiveness on supporting snapshot isolation on 1D (key-value) data. These tree-based data structures oper-

¹Silva is the Latin word for "forest," symbolizing a collection of version trees.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGSPATIAL '26, Riverside, CA, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/06

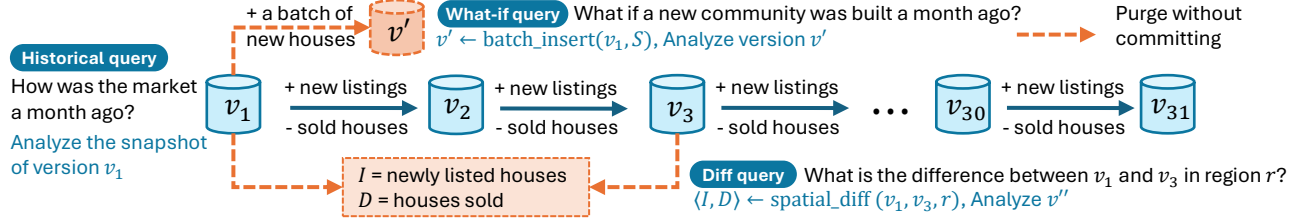


Figure 1: An illustration of Git-style version access on spatial data using an example on a real estate database. The dataset itself continuously evolves with a primary timeline of versions, updated by `batch_insert` and `batch_delete` for added and removed objects. Queries may need to investigate a historical version, create an experimental branch/sandbox (what-if query), or compare the diff between two versions.

ate similarly to their mutable counterparts, but ensure immutability by a path-copying (or copy-on-write) strategy: instead of modifying the structure in-place, an update operation copies all nodes along the affected paths (an example of our design is presented in Fig. 3). Specifically, given an update batch representing a new version, we employ a parallel algorithm that performs insertions and deletions by generating new nodes for all impacted paths, thereby leaving the preceding version entirely intact. Since the root is a common ancestor to all nodes, each update inherently produces a new root pointer, which serves as a unique entry point for accessing that specific version's snapshot. Consequently, queries on any version are as efficient as those performed on a single-version index. Since each version can be accessed in isolation from the root pointer, branching and cross-version operations can be easily expressed, making them suitable for Git-style version control. Despite these advantages and parallel algorithms proposed for the 1D functional trees, we are unaware of any work that extends the parallel FDSs to multi-dimensional spatial datasets, especially given the distinct partitioning and balancing logic required for spatial data.

To support functional spatial trees, our first attempt is to embed multi-dimensional spatial data into 1D via space-filling curves (specifically the Morton- or Z-curve) and incorporate them into an existing parallel functional BST in the CPAM library [22] (PaC-tree). We refer to this solution as the PaC-Z tree. To adapt the data structure for spatial queries, we augment each subtree with its bounding box. Our experimental results show that this simple idea already outperforms the best baseline, the MV3R-tree [68] (the state-of-the-art spatio-temporal index), while being more general by supporting arbitrary branching. To achieve higher efficiency, we observe that the embedding itself introduces overhead from multiple aspects, including computing the space-filling codes and balancing the BST via rotations, which facilitate 1D queries but not spatial ones; indeed, the bounding boxes can largely overlap in a PaC-Z tree, hence reducing the efficiency of spatial queries. To address these issues, our second and main approach is to directly design parallel algorithms with path-copying to the quad-tree **Bo**: maybe use Zd-tree? Although they are equivalent, but our method named MV-ZD, which we refer to as MVZD-tree. The key insight here is that the quad-tree is a history-independent data structure, with subtrees partitioned by spatial medians regardless of the point distributions. As such, MVZD-trees are faster than PaC-Z trees on updates due to the simpler logic of their update algorithms, and on cross-version queries due to the fully aligned spatial partitions and subspaces.

In summary, SILVA encompasses the design, analysis, and imple-

mentation of two functional trees: PaC-Z trees and MVZD-trees. Our algorithms are efficient with provable work-span guarantees, with theoretical analysis shown in ?? . SILVA enables preserving and querying any historical version, arbitrary branching from any version, and the comparison or merging of two independent versions, as well as reclaiming memory from unused parts of the indices. SILVA also supports a wide range of spatial queries, such as spatial range count and report, k -nearest neighbors, and spatial join. Accessing a snapshot takes only $O(1)$ time to retrieve the root pointer, and standard spatial queries—such as k -nearest neighbor (KNN) or range queries—can be performed on a snapshot with the same efficiency as in a single-version index. In practice, we conducted extensive large-scale scalability experiments against state-of-the-art multi-version spatial indices. The results demonstrate that our solutions outperform existing baselines by up to two orders of magnitude in processing time while requiring only one-third of the memory footprint. To summarize, the primary contributions of this paper are as follows:

- (1) We formalize the problem of spatial multi-version data processing and define the set of index operations for this problem.
- (2) We propose two spatial indexes, PaC-Z tree and MVZD-tree. Both of them support parallel algorithms with theoretical guarantees and flexible Git-style version maintenance.
- (3) We develop PaC-Z tree and MVZD-tree in the library SILVA. We have released our open-source code at [62].
- (4) We run experimental evaluations against state-of-the-art (SOTA) spatial indexes. Experiments show that SILVA significantly outperforms SOTA solutions due to better parallelism.

2 Background and Preliminaries

Parallel Computation Model. To design and analyze parallel algorithms, we use the classic *binary fork-join* paradigm [4, 15, 16] with work-span model. In this model, multiple (software) threads share memory. Each thread is a sequential RAM augmented with a *fork* instruction, which spawns two parallel child threads. The parent thread resumes execution when both children finish. A *parallel-for* can be implemented in a logarithmic number of fork steps. A computation can be viewed as a directed acyclic graph (DAG) with vertices representing computation and edges representing parent-child relationships. The work (W) of a parallel algorithm is the total number of operations (aka. the time complexity in serial execution). The *span* (or *depth*) (D) is the length of the longest dependence chain among operations. Using a randomized work-stealing scheduler, a parallel execution with work W and span D can be executed

in $W/P + O(D)$ time with probability at least $1 - W^{-c}$, for any constant $c > 0$ using P processors [15, 16, 32].

Preliminaries for Spatial Data. We describe our algorithms based on spatial data as 2D points in Euclidean space, where each point is identified by a unique ID and a location (x, y) . We note that our techniques can also be extended to shapes in low dimensions, as discussed in the supplementary material. **Yihan: Will need to double check if we still have space for that.** Many classical spatial indexes exist, including *quadrees* [27], *k-d trees* [7], and *R-trees* [6, 33, 41, 52, 61, 68, 69]. They generally work by grouping nearby points in a hierarchical structure to allow quick top-down searches.

One common indexing method is to use a *space-filling curve* (SFC). An example is the Hilbert curve, or the Z-curve [51] as shown in Fig. 2. It encodes each point as an integer along the Z-shaped curve, which effectively linearize 2D data into 1D with a total ordering. SFCs are widely used in spatial indexes [29, 56, 57, 59, 63]. Our solutions PaC-Z tree and MVZD-tree also use Z-curve.

Functional Data Structures (FDSs). Functional data structures are *immutable* data structures in which data cannot be modified in place. Instead, updates produce a new version of the structure, preserving the original version. To avoid duplicating the entire structure, a typical approach for tree-based FDSs is *path-copying* (or *copy-on-write*), where only nodes along the update path are copied, allowing unmodified nodes to be shared between versions (see examples of our designs in Fig. 2 and Fig. 3).

FDSs naturally support version access by maintaining previous states as immutable snapshots, enabling Git-style branching. For tree-based FDS, each version can be uniquely represented by the root pointer. This design ensures *atomicity* for complex updates as a transaction: if an update contains multiple operations (e.g., a batch of insertion/deletions), they become visible altogether when the root is available. It also ensures query *consistency*: once a query reads the root pointer to acquire a version, it operates on a consistent snapshot of the data, unaffected by concurrent updates.

Path-copying has been adopted by several database systems [2, 9, 28, 44, 66] for hybrid transactional and analytical processing (HTAP) workloads. Some of them are also parallel [66]. Notably, PaC-tree [22] is a state-of-the-art parallel FDS with space optimization, which our PaC-Z tree is built upon. However, these parallel data structures all focus on the general 1D key-value store. For industrial spatial indexes, the GeoGig system [31] also uses path-copying for Git-style version managements, yet its updates are sequential. To the best of our knowledge, our work is the first to introduce parallel FDS to version-aware spatial indexes.

3 Problem Definition

We first define the functions of interest for the spatial index with Git-style version control. To capture parallelism, the interface takes updates in batches and processes them in parallel; a single update is simply the special case of a batch of size one.

To track the evolving status of a spatial dataset, we use *version* to refer to a certain state of spatial index on P , and each version is represented and accessed by a *handle* (denoted by h). In SILVA, where the index is a path-copying tree structure, the handle is just the root pointer of the tree.

The Interface of a Multi-version Spatial Index. We now define

the interface for a multiversion spatial index. A multiversion spatial index should provide the *same* set of operations as a regular spatial index (e.g., construction, updates, and spatial queries), as well as version-control operations, which are useful in supporting and tracking agentic workload, such as branching, merging, and auditing. For a multi-version spatial index, a read-only query on any version should be *as easy as* querying a regular spatial index, e.g., RANGECOUNT, RANGEREPORT, KNN, and SPATIALJOIN, so we omit their detailed definition for brevity.

BUILD(P): Given a spatial dataset P , initialize a new index and return the initial version handle.

BATCHINSERT(h, I): Given a version handle h and a point set I , insert points in I into h and return the new version handle.

BATCHDELETE(h, D): Given a version handle h and a point set D , delete points in D from h and return the new version handle.

COMMIT(h, I, D): Given a version handle h , an insert point set I and a delete point set D . Return the new version handle after deleting points in D from h and inserting points in I into h .

SPATIALDIFF(h_1, h_2, r): Given two version handles h_1, h_2 , and an *axis-aligned rectangle region* r , return a pair of point sets, which represent the minimal change of points (insert or delete) within region r that transform from h_1 to h_2 .

MERGE(h_1, h_2): Given two version handles h_1 and h_2 , return the new version handle where the points in the new version are the *union* of points in h_1 and h_2 .

PURGE(h): Given an unused version handle h , delete and *free* the memory space for h , without affecting other existing versions.

While this work mainly focuses on main-memory indexes, SILVA supports load/store disk operations for durability which are described in the supplementary material.

Notice that it is straightforward to implement these operations by making a full copy of the index with each operation. However, this would be extremely inefficient due to the redundant storage of non-modified parts. The challenge is to implement these operations while reducing redundancy between index versions as shown further in the next sections.

4 The Design of the PaC-Z Tree

we build upon PaC-tree [22], a state-of-the-art 1D parallel functional search tree, by integrating it with a space-filling curve (SFC), specifically the Z-curve [51]. Below, we first overview the existing design of PaC-tree and then outline the high-level idea of our design of PaC-Z tree. Due to space limitations, we defer detailed algorithms to the appendix.

The PaC-tree. The PaC-tree [22] is a binary search tree with leaf wrapping. Structurally, each internal node maintains an entry and two child pointers, while each leaf encapsulates an array of $\Theta(B)$ entries (with B typically set to 32). Tree algorithms achieve parallelism through a divide-and-conquer concat-based algorithmic framework [11, 14, 67]. All update operations in PaC-tree support path-copying for multi-versioning and snapshot isolation. Inspired by these capabilities, our PaC-Z tree uses PaC-tree as the base design and extends it for spatial data processing.

Our PaC-Z tree. Building upon the foundation of PaC-tree, we now present our idea for PaC-Z tree. To leverage the 1D parallel

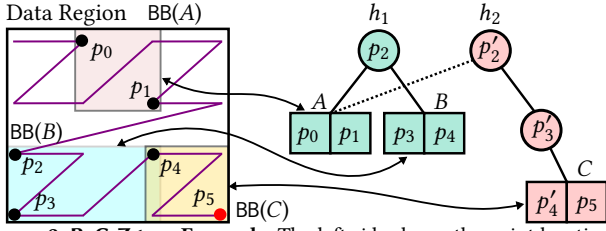


Figure 2: PaC-Z tree Example. The left side shows the point locations and bounding boxes (BB). The right side shows the structure of two versions h_1 and h_2 before and after inserting p_5 , respectively. Leaf node A is shared in this example. **Bo:** I have a new one at the end of the appendix, see Fig. 14.

update algorithms inherent in PaC-tree, we simply map each spatial data point to its corresponding Z-value, and use the Z-value as the search key for a PaC-tree. To facilitate spatial search pruning, each tree node is augmented with a bounding box covering all records in its subtree. These bounding boxes are dynamically maintained in constant time during structural changes, such as node creations or rotations. Fig. 2 illustrates an insertion process in PaC-Z tree. When point p_5 is inserted, we compute the bounding box of the new leaf node C and propagate the bounding box changes to its ancestors, p'_3 and p'_2 , by merging the bounding boxes of the two children in constant time.

The core design philosophy of PaC-Z tree is to maximize the utility of proven 1D parallel versioning algorithms. This strategy yields a simple yet highly performant spatial index with minimal implementation overhead. Crucially, it enables the direct application of PaC-tree's 1D algorithms with inherent support of parallelism and path-copying-based versioning while preserving the original theoretical bounds (see ??). For instance, tree construction, batch insertion, and deletion algorithms are reused directly, while the set-union algorithm from PaC-tree naturally facilitates our version merging logic. Built upon that, we develop algorithms for spatial-aware query algorithms, such as k nearest neighbor (k -nn) queries, range queries, and spatial-diff. Due to page limitations, we present algorithm details in the appendix.

The PaC-Z tree approach successfully provides the full versioning semantics outlined in Sec. 3. As our experiments demonstrate, it also achieves substantial performance gains over baselines like GeoGig and MV3R-tree. Nevertheless, we observed that its reliance on SFC embedding introduces inherent inefficiency. For updates, performance is hindered by the tree rotations required to maintain a strict 1D order, which is unnecessary for 2D data. For queries, SFC leads to overlapping bounding boxes among sibling subtrees. This poor spatial locality severely degrades the effectiveness of search pruning, highlighting the potential for further optimization (see the supplementary material for details). Motivated by this, we provide our main design MVZD-tree, which directly applies path-copying on the spatial index quadtree.

5 The Design of the MVZD-Tree

To overcome the rotation overhead and spatial misalignment of PaC-Z tree, we propose our main solution MVZD-tree, a path-copying quadtree-based structure. The quadtree inherently enforces strict spatial alignment, and also eliminates the need for rotations, thereby ensuring highly efficient updates. Motivated by these advantages, our goal is to design a parallel quadtree with

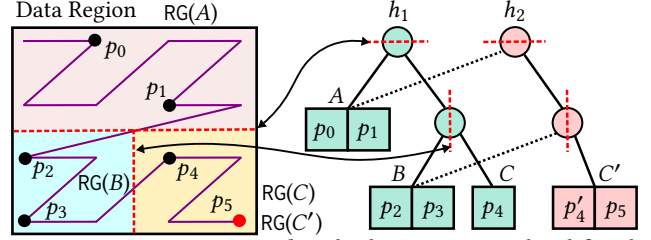


Figure 3: MVZD-tree Example. The data region is used to define the Z-curve and the partition lines. The left side shows the point locations and regions (RG). The right side shows two versions h_1 and h_2 before and after inserting p_5 , respectively. Leaf nodes A and B are shared in this example.

Algorithm 1: BUILD

Input: A point set P
Output: The initial version handle of P
Parameter: c : leaf capacity.
 b^* : maximum Z-value binary representation length.

```

1 Function BUILD( $P$ )
2   if  $P = \emptyset$  then return nullptr
3   Calculate Z-values for points in  $P$  and sort them by Z-values in parallel
4    $h \leftarrow \text{BUILDNode}(P, 0, |P|, b^*)$  //  $h$  is the version handle
5   return  $h$ 
6 Function BUILDNode( $P, l, r, b$ )
7   if  $b = 0$  or  $r - l \leq c$  then
8      $x \leftarrow$  new leaf node stores points  $\{P_l, \dots, P_r\}$ .
9     return  $x$ 
10   $\text{mid} \leftarrow$  the minimum  $i \in [l, r)$  with the  $b$ -th bit in  $P_i$  as 1
11  In Parallel:
12     $L \leftarrow \text{BUILDNode}(P, l, \text{mid}, b-1)$  // Build left subtree
13     $R \leftarrow \text{BUILDNode}(P, \text{mid}, r, b-1)$  // Build right subtree
14  Let  $x$  be a new interior node with  $L$  and  $R$  as children
15  Update subtree size for  $x$ 
16  return  $x$ 

```

version access.

While there exist parallel quadtree structures (e.g., the zd-tree [13] and P-orth tree [47]), they do not support versioning. To bridge this gap, we build MVZD-tree upon the parallel update mechanisms of the zd-tree [13] and introduce new algorithms to enable comprehensive version access, encompassing both updates and version-specific queries. Below, we first provide a high-level overview of the design of MVZD-tree, followed by a detailed discussion of the algorithms for each operation.

Overview of MVZD-Tree. Following the Zd-tree [13], MVZD-tree also employs a binary space partitioning strategy: internal nodes partition the space using the spatial median along one axis, alternating between x - and y -axes at each level. Each internal node stores its subtree size (for faster range-count queries) and two child pointers. Note that bounding boxes are *not* stored in interior nodes since they can be derived on-the-fly during traversal. Points reside in the leaf nodes, each containing an array of $\Theta(B)$ (typically 32) points ordered by Z-values. Crucially, all partition boundaries are deterministic, making the structure well-suited for multi-version access: unmodified regions can be shared across versions, improving both storage efficiency and query performance.

Operations. We now present algorithms for MVZD-tree. Throughout the paper, we assume the Z-values are b^* -bit integers.

1. BUILD. The BUILD operation, illustrated in Alg. 1, takes a list of points P as input, and build a Zd-tree over it. The algorithm begins

Algorithm 2: BATCHINSERT

Input: A version handler h , a point set I to be inserted
Output: New version handle after insertion
Parameter: c : leaf capacity.
 b^* : maximum Z-value binary representation length.

```

1 Function BATCHINSERT( $h, I$ ) // BATCHINSERT entry
2   if  $I = \emptyset$  then return  $h$ 
3   Calculate Z-values for points in  $I$  and sort them by Z-values in parallel
4   return BATCHINSERTNODE( $h, I, 0, |I|, b^*$ )
5 Function BATCHINSERTNODE( $h, I, l, r, b$ )
6   if  $h = \text{nullptr}$  then return BUILD( $I, l, r, b$ )
7    $x \leftarrow$  new tree node by copying  $h$ 
8   if  $x$  is leaf then
9     if  $b = 0$  or  $|x.\text{points}| + r \leq c + l$  then
10       merge  $\{I_l, \dots, I_r\}$  into  $x.\text{points}$  // modify copied leaf
11       return  $x$ 
12     else
13        $I' \leftarrow x.\text{points} \cup \{I_l, \dots, I_r\}$ 
14       return BUILDNODE( $I', 0, |I'|, b$ )
15    $\text{mid} \leftarrow$  the minimum  $i \in [l, r]$  with the  $b$ -th bit in  $I_i$  as 1
16   In Parallel:
17     if  $l < \text{mid}$  then  $x.\text{lc} \leftarrow$  BATCHINSERTNODE( $h.\text{lc}, I, l, \text{mid}, b-1$ )
18     if  $\text{mid} < r$  then  $x.\text{rc} \leftarrow$  BATCHINSERTNODE( $h.\text{rc}, I, \text{mid}, r, b-1$ )
19   Update subtree size for  $x$ 
20   return  $x$ 

```

by computing and sorting calculates the Z-values for all points in parallel, then recursively builds the tree via a helper function BUILDNODE(P, l, r, b), which operates on the l -th to the r -th points in P using the last b bits in their Z-values. At the top level $b = b^*$.

In BUILDNODE, if the input fits in one node (line 9), a leaf is simply returned; otherwise, it partitions $P[l..r]$ by the b -th Z-value bit (0 to the left, 1 to the right), finding the split position mid via binary search (line 10). Both subtrees are built recursively in parallel over $P[l..\text{mid}]$ and $P[\text{mid}..r]$, using the lower $b - 1$ bits. The resulting internal node x (line 14) stores the subtree size as its augmented value and is returned as the version handle.

Bo: I think we can move one method to the appendix?

2. BATCHINSERT and BATCHDELETE. MVZD-tree processes modifications in *batches* as described in Sec. 3. Due to the limited space, we introduce the BATCHINSERT algorithm here. The BATCHDELETE algorithm can be implemented similarly, and we present its details in the appendix.

The function BATCHINSERT(h, I) inserts a set of points I to a version with handle h , and returns a new version handle containing all points in h and I , keeping the original version h intact. We present the pseudocode in Alg. 2. If I is empty, h remains unchanged. Otherwise, we first calculate Z-values for points in I and sort them by Z-values in parallel. Finally, a helper function BATCHINSERTNODE(h, I, l, r, b) is called to process the l -th to the r -th points in the insertion batch I , focusing on the last b bits of their Z-values, and insert them to the subtree rooted at h .

In BATCHINSERTNODE, we first check whether h is nullptr. If so, we directly call BUILDNODE (Alg. 1) on I , and return the new subtree root on line 6. If h exists, we duplicate it into a new node x to accommodate the incoming points, leaving the original version at h intact. Note that x shares unmodified child pointers with h . If x is a leaf node (line 8), we first determine whether it has sufficient capacity for the incoming points. If so, we merge them on line 10 and return the modified leaf node x . Otherwise, we consolidate all

the points into a new sorted list I' , and then build a new subtree, returning the new handle on line 14. Alternatively, if x is an interior node, we partition the insertion batch I by the b -th bit of the Z-value. By definition, those with the b -th bit as 0 are routed to the left child, while those with 1 go to the right. Therefore, we binary search the boundary mid on line 15. Finally, we process the two children recursively in parallel, and update the size of the current interior node afterwards in line 19.

Bo: maybe we can move commit and purge operation details to the appendix?

3. PURGE. MVZD-tree supports purging an existing version without influencing other stored versions. In MVZD-tree, there is a reference counter represents how many times a tree node is used throughout the entire history. To remove a version h , PURGE recursively decrements the reference counters of tree nodes in h starting from the root. When a reference counter becomes zero, we will safely purge the tree node since it is no longer needed in any history.

4. COMMIT. COMMIT creates a single version when user commits a batch of points consists of both insertion and deletion. We assume the update batch is clean: without any point conflicts. COMMIT is implemented by a combination of BATCHDELETE, BATCHINSERT, and PURGE. It first calls BATCHDELETE to generate an intermediate version. Then, BATCHINSERT is called for the intermediate version to apply insertion. Finally, it calls PURGE to clean up the intermediate version and returns the final committed version. COMMIT pseudocode and more descriptions are presented in the supplementary material.

5. SPATIALDIFF. Given two version handlers h_1, h_2 , and a query rectangle r , SPATIALDIFF returns a pair of point sets, which stands for the minimal change of points (insert or delete) within region r that transform from h_1 to h_2 . For better clarity, we use term *spatial diff results* to denote the pair of inserted and deleted points.

As shown in Fig. 4, the algorithm processes the spatial diff by advancing two pointers simultaneously and recursively comparing the subtrees they reference. SPATIALDIFF first computes the bounding box in line 5. Since MVZD-tree partitions by spatial median, the bounding boxes can be calculated following the tree path. In addition, both pointers traverse identical paths, ensuring the bounding boxes are always aligned. This property significantly improves efficiency by enabling early pruning, as further detailed below. The remaining five cases are as follows:

Case I: identical nodes or outside query region: If the two pointers reference the same node (the region is unchanged) or bounding box does not intersect the query region r , no differences are reported in line 7. Fig. 4 shows this case by moving from root along path ①.

Case II: one nullptr: If one pointer is nullptr, either we insert points to a previously empty region, or delete all points from an existing region. Therefore, we report the other (non-null) node's points as insertions or deletions accordingly in line 8 and line 10.

Case III: two leaves: If the two pointers reference difference leaf nodes, SPATIALDIFF computes the inserted/deleted points for the points in two leaves by scanning in line 12. Fig. 4 demonstrates this case by following the path ② $\rightarrow$ ④. D and D' are two leaf nodes and we need to scan them to obtain the inserted/deleted points. Since all points are sorted, we can achieve this by linear scanning.

Case IV: one leaf, one interior: If two pointers refer to a leaf node

Algorithm 3: SPATIALDIFF

Input: Two version handlers h_1, h_2 , and a query rectangle r
Output: The insertion, deletion point sets can transform h_1 to h_2
Parameter: b^* : maximum Z-value binary representation length.

```

1 Function SPATIALDIFF( $h_1, h_2, r$ )
2    $\langle I, D \rangle \leftarrow \text{SPATIALDIFFNode}(h_1, h_2, r, b^*)$ 
3   return  $\langle I, D \rangle$ 
4 Function SPATIALDIFFNode( $h_1, h_2, r, b$ )
5    $\text{box} \leftarrow$  current tree node bounding box
6    $I \leftarrow \emptyset, D \leftarrow \emptyset$ 
7   if  $h_1 = h_2$  or  $\text{box} \cap r = \emptyset$  then return  $\langle \emptyset, \emptyset \rangle$  // Case I: identical nodes
8   else if  $h_1 = \text{nullptr}$  and  $h_2 \neq \text{nullptr}$  then // Case II: one nullptr
9      $I \leftarrow$  range report results of  $h_2$  in query region  $r$ 
10    else if  $h_1 \neq \text{nullptr}$  and  $h_2 = \text{nullptr}$  then
11       $D \leftarrow$  range report results of  $h_1$  in query region  $r$ 
12    else if  $h_1$  is leaf and  $h_2$  is leaf then // Case III: two leaves
13       $\langle I, D \rangle \leftarrow$  spatial diff results for  $h_1$ .points and  $h_2$ .points in  $r$ .
14    else if  $h_1$  is leaf and  $h_2$  is not leaf then // Case IV: one leaf, one interior
15       $\langle I, D \rangle \leftarrow$  spatial diff results for  $h_1$ .points and  $h_2$  in  $r$ 
16    else if  $h_1$  is not leaf and  $h_2$  is leaf then
17       $\langle I, D \rangle \leftarrow$  spatial diff results for  $h_1$  and  $h_2$ .points in  $r$ 
18    else
19      In Parallel: // Case V: two interiors
20       $\langle I_L, D_L \rangle \leftarrow \text{SPATIALDIFF}(x.lc, y.lc, r, b-1)$ 
21       $\langle I_R, D_R \rangle \leftarrow \text{SPATIALDIFF}(x.rc, y.rc, r, b-1)$ 
22       $\langle I, D \rangle \leftarrow \langle I_L \cup I_R, D_L \cup D_R \rangle$ 
23    return  $\langle I, D \rangle$ 

```

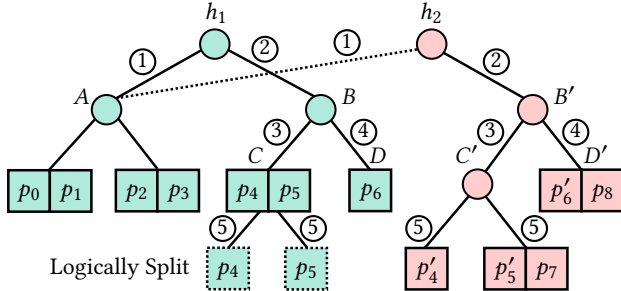


Figure 4: MVZD-tree Spatial Diff Example. h_1 and h_2 are two version handles. The numbers on the tree path indicate different spatial diff cases.

and an interior node respectively, we continue to traverse down to the leaf nodes on interior node side and comparing them to the list of points in the leaf node from lines 14-17. As we traverse down the interior node, we (logically) split the set of points in the leaf node to match. Note that the leaf node partition does not make structural changes, and the partitioning does not involve any data movement since points are sorted. In example Fig. 4, when the two pointers traverse along tree path ② $\rightarrow$ ③. C is a leaf node in version h_1 and C' is an interior node in version h_2 . We continue traversing the interior node side C' , and logically split the leaf node side C . Finally, we reach the leaf node under C' and can obtain the results. *Case V: two interiors:* SPATIALDIFF handles the two interior nodes case in line 19 by recursively calling SPATIALDIFFNode for the child pointers of the two interior nodes. Since the two subtrees are aligned, we only need to compare the children in the same direction, i.e., left-to-left and right-to-right. Fig. 4 shows this case by following path ② $\rightarrow$ ③ and ② $\rightarrow$ ④.

Finally, the spatial diff results are merged and returned in line 22.

6. MERGE. Given two version handlers, h_1 and h_2 , and a conflict-solving strategy S , the MERGE function returns a new version handle

Algorithm 4: MERGE

Input: Two version handles h_1, h_2 , and conflict resolving strategy S .
Output: The handler of new merged version.
Parameter: r_g : The global region (entire space).

```

1 Function MERGE( $h_1, h_2, S$ ) // MERGE entry
2    $h_b \leftarrow$  common base version handle of  $h_1$  and  $h_2$ 
3   In Parallel:
4      $\langle I_1, D_1 \rangle \leftarrow \text{SPATIALDIFF}(h_b, h_1, r_g)$  // get changes from  $h_b$  to  $h_1$ 
5      $\langle I_2, D_2 \rangle \leftarrow \text{SPATIALDIFF}(h_b, h_2, r_g)$  // get changes from  $h_b$  to  $h_2$ 
6      $\langle I, D \rangle \leftarrow$  no conflict updates by linear scan  $I_1, I_2, D_1, D_2$ 
7      $\langle I', D' \rangle \leftarrow$  conflict updates by linear scan  $I_1, I_2, D_1, D_2$ 
8      $h_t \leftarrow h_b$ 
9     if strategy  $S$  is provided then // solve conflict
10       $h_t \leftarrow \text{Commit } I', D'$  to  $h_t$  according to  $S$ 
11       $h \leftarrow \text{COMMIT}(h_t, I, D)$  // commit not conflict points
12      PURGE( $h_t$ )
13    return  $h$ 

```

containing all points in h_1 and h_2 . The spatial merge operation is implemented based on SPATIALDIFF and COMMIT. Firstly, we calculate the common base version h_b for h_1 and h_2 in line 2. The base version can be achieved by tracking back along the version history. e.g., when versions are managed in tree-like style, we can get h_b by their *least common ancestor*.

Then we call SPATIALDIFF to compute the diff of the base version to both h_1 and h_2 in line 4 and line 5, respectively. After that, we merge the modification sets together in line 6 to obtain the non-conflict and conflict modifications, respectively. Since all points are sorted by Z-values, this procedure can be achieved by linear scanning the point sets. If a conflict-solving strategy is provided, we will commit the conflict updates according to the given strategy and obtain an intermediate version in line 9. Finally, we call COMMIT to commit the non-conflict changes in line 11, purge the intermediate version, and return the new merged version handle. Note that there is no extra space usage except for the merged version.

6 Theoretical Analysis

This section presents the work and span analysis for version operations in SILVA: BUILD, BATCHINSERT, BATCHDELETE, and COMMIT. We omitted spatial queries and merge since they are highly data sensitive and the analysis on the worst case becomes trivial. Since all three indexes use *leaf wrapping* technique, we use B (usually 32) to represent the leaf node capacity and assume it is a constant.

6.1 PaC-Z Tree

THEOREM 6.1 (BUILD COST). *Given a set of points P with size n , PaC-Z tree can be built in $O(n \log n)$ work with $O(\log n)$ span.*

Proof. The build procedure first compute Z-values for all points in $O(n)$ work with $O(\log n)$ span, then sort them in $O(n \log n)$ work with $O(\log n)$ span (using optimal parallel comparison-based sorting), and finally build a perfect balanced binary search tree use divide-and-conquer manner in $O(n)$ work with $O(\log n)$ span. Therefore, build work is $O(n + n \log n + n) = O(n \log n)$, and build span is $O(\log n + \log n + \log n) = O(\log n)$. $\square$

LEMMA 6.2 (BATCH INSERT/DELETE COST). *Given a PaC-Z tree T built upon point set P with size n , and a point set P' with size m . Batch insert/delete P' to/from T can be processed in $O(m \log n)$ work with $O(\log m \log n)$ span.*

Proof. We first analyze batch insertion first, and the analysis of batch delete is similar. The batch insert procedure contains three parts: 1) partition P' into subtrees until meet the base cases; 2) process base cases (append inserted points to existing leaf; dealing with leaf overflow); and 3) rebalance after insertion. Firstly, node copy for a tree node has $O(1)$ cost. Since both interior and leaf nodes have constant number of fields. For the partition part, T has $O(\log n)$ levels, and in each level the partition procedure cost $O(m)$ work with $O(\log m)$ span. Therefore, the partition work in total is $O(m \log n)$ and the partition span is $O(\log m \log n)$. For the base case, we need to touch all updated points in the worst case. Therefore the work is $O(B + m) = O(m)$ and span is $O(\log(B + m)) = O(\log m)$. For the rebalance part, PaC-Z tree uses concat primitive [67] for rebalancing. Each concat can combine two trees with size x and y in $O(\log \frac{x}{y})$ work and span ($x > y$). There are $O(m)$ leaf nodes need to be rebalanced, and thus $O(m)$ concat will be called. Each concat costs at most $O(\log \frac{n}{m})$ work and span (w.l.o.g. assume $n > m$). Therefore the total work for rebalance is $O(m \log \frac{n}{m})$, and the span is $O(\log \frac{n}{m})$.

Adding up the three parts, the insertion work is $O(m \log n + m + m \log \frac{n}{m}) = O(m \log n)$, and insertion span is $O(\log m \log n + \log m + \log \frac{n}{m}) = O(\log m \log n)$, bounded by partition. $\square$

THEOREM 6.3 (COMMIT COST). *Given a PaC-Z tree T build upon point set P with size n , delete point set D with size m_1 , and insert point set I with size m_2 . A new version T' can be committed in $O((m_1 + m_2) \log n)$ work with $O(\log(m_1 m_2) \log n)$ span.*

Proof. Commit consists of three steps. Batch deletion, batch insertion, and purge the intermediate version. First we use Lem. C.2 to get the costs for batch insertion and batch deletion. For batch deletion, we create a temporary version T_t by deleting D from T , which costs $O(m_1 \log n)$ work and $O(\log m_1 \log n)$ span. Then we batch insertion I into T_t . Since m_1 points are deleted from P , the size of T_t is $n - m_1$. Therefore the batch insertion costs work $O(m_2 \log(n - m_1))$ and span $O(\log m_2 \log(n - m_1))$. Finally, we purge T_t , which contains $n - m_1$ points. The work and span for purge are the same as traverse all tree nodes in parallel, i.e., $O(n - m_1)$ work and $O(\log(n - m_1))$ span.

Combine all three parts together, we obtain that the commit procedure costs with $O(m_1 \log n + m_2 \log(n - m_1) + (n - m_1)) = O((m_1 + m_2) \log n)$ work and $O(\log m_1 \log n + \log m_2 \log(n - m_1)) = O(\log(m_1 m_2) \log n)$ span. $\square$

6.2 MVZD-Trees

Given a spatial dataset P with n points, let $d_{\max}$ ($d_{\min}$) be the furthest (closest) pairwise distance in P , respectively. The aspect ratio of P , denoted by Δ_P , is defined to be $\Delta_P = d_{\max}/d_{\min}$.

THEOREM 6.4 (BUILD COST). *Given a set P with n points, a MVZD-tree can be built on P in $O(n \log n)$ work with $O(\log n \log \Delta_P)$ span.*

Proof. The first step in the build algorithm is to compute and sort by Z-values which requires $O(n \log n)$ work and $O(\log n)$ span when using optimal parallel comparison-based sorting.

To compute the cost of the tree construction step, we observe that MVZD-tree is a binary tree with $O(n)$ leaf nodes (each leaf contains at least 1 point), and the total number of tree nodes is $O(n)$. For each tree node, a binary search with cost $O(\log n)$ is performed

to partition the points (see line 10). Therefore, the total work is $O(n \log n)$. For the span, the height of MVZD-tree is determined by the aspect ratio Δ_P . Since the space is halved for each level, the height of MVZD-tree is $O(\log \Delta_P)$. For each level of recursion, a binary search is performed to partition the points into two parts, therefore the span is $O(\log \Delta_P \log n)$. $\square$

LEMMA 6.5 (BATCH INSERT/DELETE COST). *Given a MVZD-tree T maintaining dataset P with size n , and a point set P' with size m . Batch inserted/deleted P' to/from T can be processed with $O(m \log \Delta_P)$ work and $O(\log \Delta_P \log m)$ span.*

Proof. We show the proof of batch insertion here, and the proof for deletion is similar. Firstly, the tree after inserting P' into P has the same shape as directly constructing a MVZD-tree on top of $P \cup P_I$ since MVZD-tree is partitioned using spatial median — the shape is determined once the global region is provided. Given the fixed node size, the node copy procedure costs $O(1)$ operations. Therefore the tree height after batch insertion is $O(\log \Delta_{P \cup P'})$.

We analyze the work first. The tree height of T is $O(\log \Delta_P)$. For each level of the tree, we need to partition entire P' once. The partition procedure has $O(m)$ work and $O(\log m)$ span. Therefore, the total work of batch insertion is $O(m \log \Delta_P)$.

For the span part, consider the longest path during the insertion. It touches $O(\log \Delta_P)$ tree nodes (a chain from root to leaf), and for each touch it requires a $O(\log m)$ binary search to partition (as shown in line 15). Therefore, the span is $O(\log \Delta_P \log m)$. $\square$

THEOREM 6.6 (COMMIT COST). *Given a MVZD-tree T built for point set P with size n , delete point set D with size m_1 , and insert point set I with size m_2 . A new version T' can be committed in $O((m_1 + m_2) \log \Delta_P)$ work with $O(\log m_1 m_2 \log \Delta_P)$ span.*

Proof. We consider the three steps for commit: 1) Batch delete D ; 2) Batch insert I ; and 3) Purge the intermediate version.

For the first step, we create a temporary version T_t by batch deleting D , which costs $O(m_1 \log \Delta_P)$ work with $O(\log \Delta_P \log m_1)$ span according to Lem. C.5. T_t is the same shape as directly built on point set $P \setminus D$. Therefore, the second step costs $O(m_2 \log \Delta_{P \setminus D})$ work and $O(\log \Delta_{P \setminus D} \log m_2)$ span. To purge T_t , we just need to traverse T_t in parallel with $O(n - m_1)$ work and $O(\log \Delta_{P \setminus D})$ span. Since $|P \setminus D| < |P|$, then $O(\log \Delta_{P \setminus D} \log m_2) = O(\log \Delta_P \log m_2)$.

By combining the three parts, the commit costs $O(m_1 \log \Delta_P + m_2 \log \Delta_{P \setminus D} + (n - m_1)) = O((m_1 + m_2) \log \Delta_P)$ work and $O(\log \Delta_P \log m_1 + \log \Delta_P \log m_2 + \log \Delta_P) = O(\log m_1 m_2 \log \Delta_P)$ span. $\square$

7 Experiments

7.1 Summary of Experimental Results

We conduct extensive experiments on both real-world [54] and synthetic datasets [30] to illustrate the efficiency of SILVA in maintaining spatial indexes with versioning. Compared with MVR-tree [43], SILVA achieves up to $82\times/84\times$ speedup in build time when running sequentially, while only using 25.6%/30.5% memory space on synthetic/real-world datasets, respectively. For batch insertion and batch deletion, SILVA can achieve up to $145\times$ speedup compared with existing baselines when running sequentially. For spatial queries, MVZD-tree achieves the best or is comparable to the best performance among all tested methods, which includes the well-recognized single-version R-tree in *boost* [17]. PaC-Z tree shows

slightly worse performance than MVZD-tree due to its spatial misalignment. For spatial diff query, MVZD-tree takes full advantage of spatial invariance, which is up to 17× faster compared with other baselines for small amounts of modifications. SILVA also achieves high parallelism, exhibits linear scalability with respect to the number of processing cores.

In conclusion, SILVA outperforms existing multi-version spatial indexes significantly in terms of build, batch insertion/deletion, spatial queries, and incurs a much smaller memory footprint.

7.2 Experimental Setup

Setup. All experiments were conducted on a 96-core machine with four-way Intel Xeon Gold 6252 CPUs and 1.5 TB of memory. All indexes are implemented in C++ and compiled by g++ 14.2.1 with `-O3` flag. We use parallel primitives from PARLAYLIB [12], and `numactl -i all` for parallel experiments, which interleaves memory across CPUs on NUMA architectures. Results are reported as the average of three test rounds after a warm-up run.

Baselines. We test SILVA against existing approaches: MVR-tree, MV3R-tree, and R-tree. All these baselines are sequential. MVR-tree and MV3R-tree are the state-of-the-art existing multi-version spatial indexes proposed by Tao and Papadias [68]. MV3R-tree maintains an extra 3D R-tree compared with MVR-tree to accelerate cross-version queries. We include MV3R-tree in our evaluation since it serves as a complete solution. We tested both MVR-tree and MV3R-tree as they are the SOTA multiversion spatial indexes. They only support modify the latest version, and thus Git-style operations (e.g., branching) are not supported. We use MVR-tree implementation in *libspatialindex* [43]. Since its original code is not available, we implement MV3R-tree based on *libspatialindex*. boost-R is a well-recognized open-sourced single-version R-tree implementation in *Boost Geometry Library* [17]. We include boost-R with quadratic split in our experiments to support our claim that SILVA does not pay an extra cost for single-version query efficiency, despite adding support for versions. Leaf capacities of all approaches are set to 32.

Dataset. We run experiments on both real-world and synthetic data. For real-world data, we use *Open Street Map* (OSM) [54]. For synthetic data, we generate using Uni form and skewed Varden [30] distributions with coordinate range $[0, 10^7]$. The statistics of used datasets are discussed at the beginning of each evaluation section, and a summary of them can be found in the supplementary material.

7.3 End-to-end Performance with a Case Study

To evaluate the end-to-end performance with a real-world workload, we used the yearly commits from the *OSM Bhutan* [54] dataset. The detailed dataset statistics are shown in the supplementary material. We started with the initial dataset from 2018 and simulated the history of the yearly changes until 2025. All changes in one year are applied as one batch commit with a mix of insertions, deletions, and updates. Note that there are sudden bursts of point modifications in year 2022 and 2023, which contain more modified points than existing points before 2018. We measured both time cost and memory usage for each COMMIT function call. The yellow bars show the number of modified points in each data collection year. We also show the sequential running time of SILVA since the

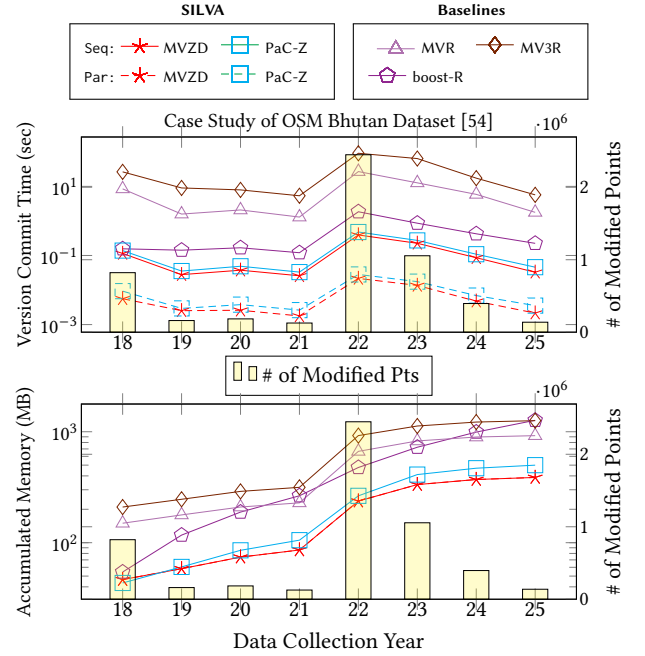


Figure 5: Real-world case study on Version Commit Time and Memory Usage (OSM Bhutan from 2018-2025), lower is better. Solid lines are measured sequentially. Dashed lines are measured in parallel (96-cores). Bars show the number of modified points in each year.

baselines are all sequential.

Time Cost. The upper part of Fig. 5 shows the commit time in seconds (log-scale) for each version. For all tested indexes, the version commit time is proportional to the batch size, as expected.

The sequential version of SILVA already consistently outperforms all baselines by up to 68×. boost-R is slightly faster than MV3R-tree and MVR-tree, but it does not support versions. When running in parallel (96 cores), MVZD-tree, PaC-Z tree achieve on average 986×, 690× speedup compared with MVR-tree, and on average 69×, 49× speedup compared with boost R-tree. In SILVA, MVZD-tree has the best overall performance since it does not need rebalance as mentioned in Sec. 5.

Memory Usage. The lower part of Fig. 5 shows the accumulated memory in MB (log-scale) to maintain all history versions. Boost R-tree has to deep copy the previous version, and thus its memory growing regardless of whether the number of modified points is large or small (e.g., from 2018 to 2019). Such high memory overhead makes it less competitive than others in supporting versions. For this reason, we exclude boost-R in some of the future experiments.

All other techniques are version-aware, with memory increasing proportionally to the batch size of modified points. MVR-tree and MV3R-tree consume more memory due to their large node sizes [43]. PaC-Z tree has the lowest memory usage in 2018, since it is perfectly balanced. MVZD-tree consistently achieves the smallest memory footprint in years beyond 2018. This is because MVZD-tree does not modify a tree node unless changes apply to its subtree. In contrast, PaC-Z tree needs to copy more tree nodes due to tree rebalance and spatial misalignment. As such, MVZD-tree is superior than tested methods in terms of memory footprint for version management.

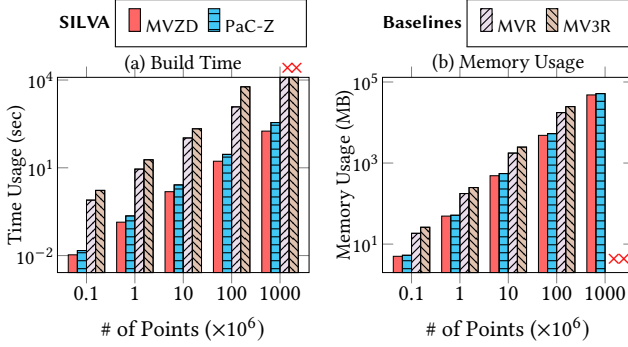


Figure 6: BUILD time (Sequential) and memory evaluation on Varden dataset, lower is better (log-scale). X means timeout or not applicable.

7.4 Static Construction

We now provide an in-depth study of the performance of the construction time of an index, with the results given in Fig. 6. Since the baselines are sequential, we also report our sequential time for comparison. We show the results for Varden datasets here since we obtained similar results for Uniform dataset. Please refer to the supplementary material for more details.

Time Cost. As shown in Fig. 6 (a), even sequentially, SILVA consistently outperforms the SOTA multiversion spatial indexes (MVR-tree and MV3R-tree) by around two orders of magnitude. The reason is that SILVA have highly efficient construction algorithms, while MVR-tree and MV3R-tree can only be built incrementally (i.e., no bulkload supported), which causes this significant gap in performance. Consequently, MVR-tree and MV3R-tree timed out (> 3 hours) on the dataset with 10^9 points. In SILVA, MVZD-tree achieves consistently the best construction performance, since unlike PaC-Z tree, MVZD-tree avoids the overhead of calculating and storing extra information (such as MBR and Z-value).

Memory Usage. As shown in Fig. 6 (b), MVZD-tree/PaC-Z tree use on average 72.5%/70.2% less memory than MVR-tree, respectively. This is because nodes in MVR-tree and MV3R-tree [43] are much larger: it needs to store start/end timestamps, low/high corners for points, augmented bounding box for each tree node, and data pointers, etc. It results in about four times larger tree node size compared with MVZD-tree.

In SILVA, PaC-Z tree uses more memory (although it is balanced) since it stores extra information such as Z-values and MBRs to locate points and accelerate spatial queries. In addition, despite MVZD-tree not achieving the best in single-version memory usage, it has the smallest accumulated memory usage when handling multiple versions, as we show in the case study Fig. 5 mentioned above.

7.5 Batch Insert/Delete, Commit, and Merge

BATCHINSERT/DELETE. We start with a fixed index of 100M points and perform a batch insert/delete with batch sizes ranging from 10^4 to 10^8 (batches smaller than 10^4 take negligible time). Due to space limitation, we only show Varden results. The performance for Uniform is similar and given in the supplementary material.

Fig. 7 shows the insertion and deletion times on the Varden dataset. Although SILVA performs comparably to the baselines for small updates, it becomes significantly faster when the batch size

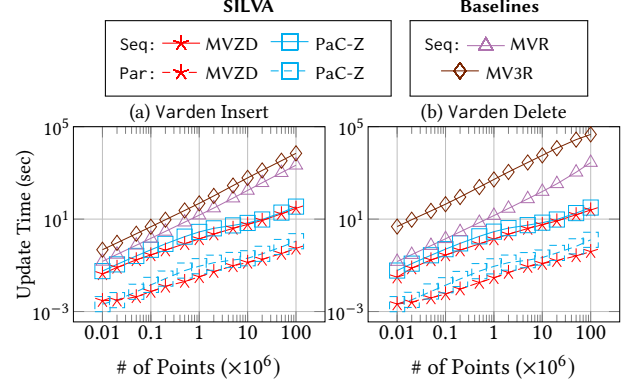


Figure 7: Batch update evaluation on Varden with 100M points, lower is better (log scale). Dashed lines are running in parallel (96 cores).

increases. This is because MVR-tree and MV3R-tree cannot take advantage of batch updates and must update points one at a time. In contrast, SILVA has designated batch update algorithms (see Sec. 4 and 5), leading to the increasing gap as the batch size becomes larger. When leveraging 96-core parallelism, MVZD-tree and PaC-Z tree are up to three orders of magnitude faster than MVR-tree, and up to $12,971\times$ faster than MV3R-tree. In SILVA, MVZD-tree is consistently the fastest, since it avoids tree-balancing overhead and benefits from more shared nodes.

COMMIT and MERGE. As shown in Fig. 8 (a), we evaluate the COMMIT operation under a mixed workload (50% insertions, 50% deletions). Since COMMIT consists of BATCHINSERT/DELETE followed by a cheap PURGE operation, they have similar performance as batch updates. Note that MVZD-tree is the fastest in SILVA, and we did not include other baselines since SILVA already proves its batch update efficiencies in the above discussion.

For the MERGE operation, we reuse the COMMIT workload to create two versions (one for insertions, one for deletions). The MERGE essentially consists of a diff and a commit operation. One interesting observation is that MVZD-tree is two orders of magnitude faster than PaC-Z tree for a small number of changes. This is due to the efficient spatial diff algorithm that advances two pointers simultaneously to quickly prune large portions of unchanged regions. In contrast, PaC-Z tree has a significant overhead in collecting all changes in the two versions, even with very small batch sizes. In practice, users tend to use the MERGE operation when there are only a few changes, which is what the algorithm in MVZD-tree is optimized for. As the modification eventually affecting all points, MVZD-tree pruning improvement becomes negligible, and the performance of all methods converges. MVZD-tree takes slightly longer due to the overhead of spatial diff algorithm as detailed below.

7.6 SPATIALDIFF

To evaluate the SPATIALDIFF function, we first COMMIT a new version with the same workload as Sec. 7.5. Then, we generate random query regions and classify them based on the number of points they contain prior to the commit: $[0, 100)$ for *small* regions and $[100, 10,000)$ for *medium* regions. We exclude large regions since the cost will be the same as scanning the entire dataset. Due to the limited space, we show Varden results in Fig. 9, and please refer to

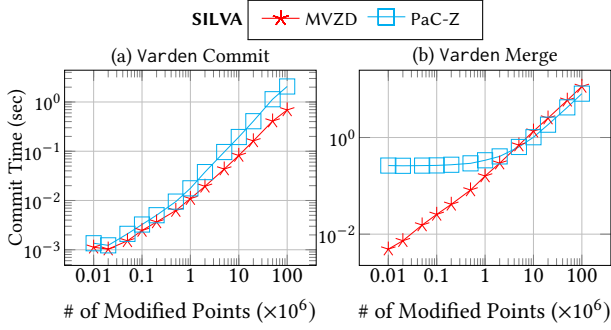


Figure 8: Commit and Merge time on Varden with 100M points, lower is better. Modification consists of 50% insertion and 50% deletion.

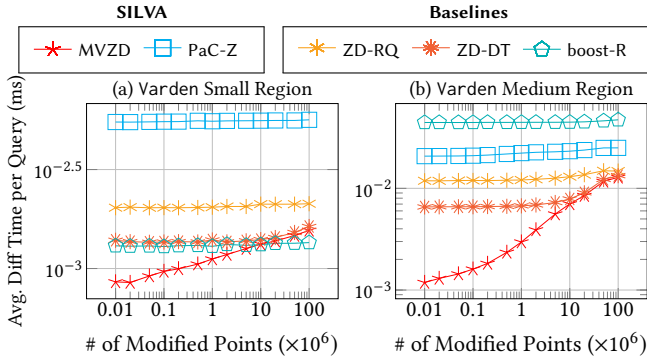


Figure 9: Spatial diff performance on Varden with 100M points, lower is better. Modified points consist of 50% insertion and 50% deletion.

the supplementary material for UniForm results.

To evaluate the effectiveness of spatial diff in MVZD-tree, we compare it against three baselines. The first two, namely ZD-RQ and boost-R, represent the naive approach using single-version indexes. They call range report first on the two versions (using Zd-tree and R-tree, respectively), and compute the difference by post-processing. The third baseline, ZD-DT, is designed to show the performance gains of Zd-tree structural sharing. It employs the same spatial diff algorithm (as presented in Alg. 3), but operates on two entirely independent Zd-trees without any shared nodes.

Fig. 9 shows that MVZD-tree is the clear winner for small modifications, i.e., less than 10%, and remains highly competitive for larger changes. This validates the effectiveness of our proposed spatial diff algorithm (Alg. 3). The comparison between ZD-DT to ZD-RQ shows the effectiveness of the dual tree traversal algorithm even without node sharing. Interestingly, boost-R performs well for small regions where the range report queries are lightweight that are quick to compare. However, it becomes the slowest for medium regions since the query results become significantly larger. Finally, PaC-Z does not perform well on spatial diff queries, due to overlapping bounding boxes introduced by the Z-curve layout, which limits the effectiveness of pruning during range reporting.

To conclude this part, MVZD-tree has the overall best performance of spatial diff queries, especially for detecting differences for the number of modifications is small.

7.7 Parallel Speedup

We now evaluate the self-relative speedup of SILVA indexes across different core counts. We use a dataset with 10^8 points with 10% modifications (for batch insert/delete, commit, and merge) the same as Sec. 7.5. Due to the space constraints, we focus on *build* and *merge* operations here, and results for other operations are detailed in the supplementary material.

As shown in Fig. 10 (a), all SILVA indexes achieve linear speedup in terms of building time as the number of cores grows to 48. MVZD-tree continues to achieve high speedup due to its minimal tree node size design. For merge, Fig. 10 (b) shows that MVZD-tree has a slightly lower speedup with a large number of threads due to the cost of parallel memory allocation in its traversal.

In a word, all SILVA indexes exhibit excellent parallel scalability.

7.8 Standard Spatial Queries

SILVA provides snapshot isolation: each version can be treated as a single-version spatial index. Therefore, existing single-version spatial queries can be seamlessly ported to SILVA without any additional overhead. We implemented *range report*, *range count*, *kNN*, and *spatial join* for SILVA to demonstrate its extensibility.

Fig. 11 (a) shows a scatter plot of *range report* times with respect to the query output size. Even though SILVA indexes provides access to all historical versions, their query performance is competitive with the highly optimized boost-R. This shows that SILVA does not have to trade off in performance to provide all its unique features.

Fig. 11 (b) presents similar results for the *range count* queries. We exclude MVR-tree and R-tree from this evaluation since they lack specialized range count algorithms and essentially use range report. In contrast, MVZD-tree and PaC-Z tree achieve 92× and 10× self-speedup respectively compared with range report due to subtree size augmentation. Note that MVZD-tree obtains significant self-speedup due to its perfect spatial alignment.

Fig. 11 (c) shows the results of the *kNN* queries while varying k from 1 to 100. MVZD-tree is comparable to, or even faster than the highly optimized boost-R. Both MVZD-tree and boost-R achieve at least an order of magnitude speedup over other techniques.

For MVZD-tree, we also implemented a spatial join (report) operation based on *synchronized traversal* [35], which finds all pairs of points across two versions that fall within a specified distance bound. We prepare two versions with 10^6 points (in Varden distribution), setting the distance bound to 50,000. Fig. 11 (d) plots the spatial join time as the input size increases. MVZD-tree can process a spatial join report in 2 seconds, which shows its extensibility in supporting existing spatial queries.

7.9 Real-world Data Evaluation

To verify the generalization of our results, we evaluated essential operations (including build, commit, merge) using up to 100M records from the OpenStreetMap (OSM) real-world dataset.

Fig. 12 (a) shows that SILVA maintains its performance advantage over MVR-tree, achieving up to two orders of magnitude speedup during index construction. Fig. 12 (b) presents the construction memory footprint. Due to its compact design, SILVA significantly reduces memory consumption compared to MVR-tree. Although MVZD-tree incurs a slightly larger initial footprint, as

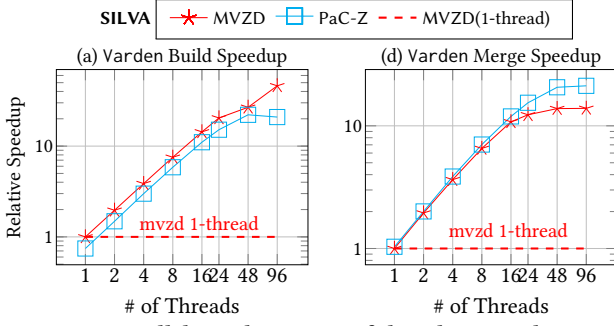


Figure 10: Parallel speedup w.r.t. # of threads on synthetic Varden datasets with 100M points, higher is better. The relative speedup is calculated w.r.t. MVZD tree 1-thread (shown in the dashed horizontal line) performance.

shown in Sec. 7.3, it conserves space during subsequent updates.

Fig. 12 (c) and (d) show the performance of commit and merge operations on the OSM Japan dataset (with approximately 100M points). Following our earlier setup, we select modification subsets of increasing sizes, splitting each into 50% insertions and 50% deletions. The results are consistent with our previous findings shown in Fig. 8.

To summarize, SILVA is equally efficient on real-world datasets as it is on synthetic data.

8 Related Work

This section reviews the related work on traditional spatial indexes, multiversioning for concurrency control, non-spatial versioning approach, multiversion spatial indexes, and functional data structures.

Single-version Spatial Indexes. Spatial indexes have played a crucial role in database systems in both academia [1, 25, 26] and industry [49, 50, 55, 58, 63] for decades. Traditional spatial indexes, such as *quadtrees* [27], *k-d trees* [7], *segment trees* [8], and *R-trees* [6, 33], are designed to accelerate spatial queries over a single dataset sequentially. To leverage multi-core architectures, several studies have introduced parallelism to spatial indexes, including bulk-loading operations for *R-trees* [3], *parallel R-trees* [36], and parallel construction and batch updates for *Zd-trees* [13], *k-d trees* [48], and *Orth trees* [47]. In addition, 1D indexes can be extended to manage spatial data via space filling curves, for example, *UB-tree* [59] use Z-curve on top of B+-tree. However, all these approaches are limited to single-version operations, providing only a single global view of the index for all queries and updates. Furthermore, they rely on in-place updates to achieve efficiency, which inherently conflicts with the concept of the Git-style versioned access that our work focuses on.

Multi-Version Concurrency Control (MVCC). With the rise of main-memory databases, recent studies have replaced traditional lock-based concurrency control with latch-free approaches to achieve high parallelism [19, 38, 40, 44]. The key idea is to use version chains or copy-on-write (CoW) to modify parts of the index, thereby avoiding in-place updates. However, these techniques are designed to support high-throughput transactions, and they do not maintain previous versions for historical access. Although MVCC can be extended to maintain historical versions, their main focus

is the concurrent update of the latest version, which is essentially different from our design goal.

Non-spatial Versioning Approach. The idea of non-spatial versioning starts from *hypothetical databases* [70], which construct virtual indexes that temporarily store only the modified records to support what-if analysis. *Bw-tree* [42] support versioning via appending delta logs to tree nodes. Recent systems, like *DataHub* [10], *Decibel* [46], and *OrpheusDB* [34] have introduced Git-style version control for relational tables. However, these systems are not designed to natively support spatial indexing. Consequently, executing both single-version and cross-version spatial queries are highly inefficient.

Multiversion Spatial Indexes. There have been several explorations of multi-version spatial indexes in the literature. For instance, Kolovson and Stonebraker [39] manage versions using a combination of *segment trees* and *R-trees*. The *historical R-trees* [52] extended the **path-copying** technique to *R-trees* to support multiversion spatial data. The *MV3R-tree* [68] combines an *MVR-tree* (an *R-tree* variant for *MVB-tree* [5]) and a *3DR-tree* (which treats time as the third dimension). In this structure, *MVR-tree* is used for historical single-version access, while *3DR-tree* is used for cross-version queries. Although these methods provide some level of multiversion access, all updates are limited to the latest version. They do not support advanced Git-style version control, such as *branching* and *what-if* analysis [18].

Functional Data Structures. A more structured way to support Git-style versioning is through functional data structures. Parallel Augmented Map (PAM) [67] maintains a multi-version map using a functional binary search tree with concat-based algorithms [14]. Parallel Compressed tree (PaC-tree) [22] further improves on this idea by wrapping tree leaf nodes into compressed blocks to save space without significantly sacrificing update and query performance. PAM was also used to build a multicore in-memory HTAP DBMS [66] that achieves snapshot isolation. However, none of the work above achieves full versioned access on spatial data, including both query and update.

Relation to SILVA. SILVA is the *first parallel spatial index library* to leverage functional data structures for Git-style versioning. In contrast to single-version spatial indexes, SILVA persistently maintains all historical versions. Unlike existing MVCC mechanisms, it supports flexible, Git-style branching and merging. Compared with adapting 1D indexes via space-filling curves, SILVA achieves highly efficient spatial query performance due to its native spatial design. In addition, SILVA demonstrates that the functional data structure paradigm is highly effective for spatial data, especially combine with strict spatial alignment (the MVZD-tree). Last but not least, SILVA consistently outperforms the state-of-the-art multiversion spatial indexes.

9 Conclusion

This paper defines the versioned spatial data access problem, and proposes SILVA, a parallel spatial index library with versioned access using the idea of functional data structures. SILVA supports both single- and multi-version spatial data management, both with high parallelism. On the one hand, accessing any specific version in SILVA is as easy as accessing a single-version index without

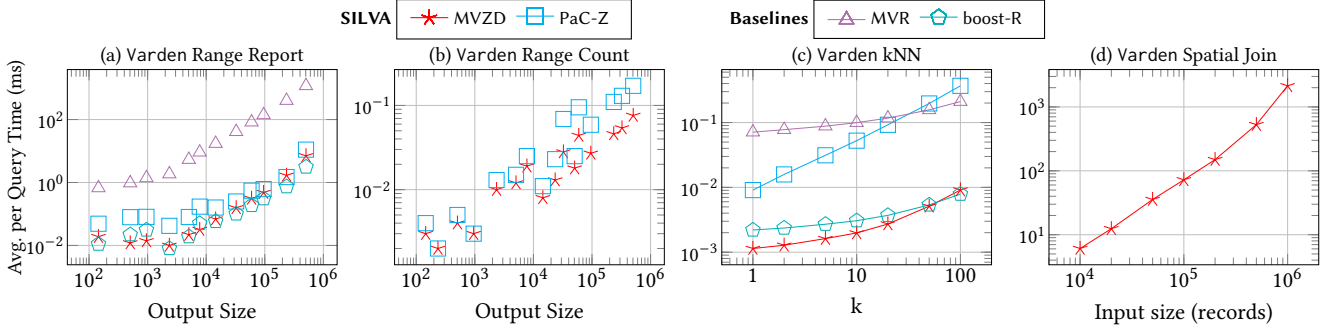


Figure 11: Single-version query (sequential) evaluation on synthetic datasets with 100M points, lower the better. x-axis is the parameter studied (i.e., # of points, output size, or k). All axes are in log scale. For (a), # of points is chosen before modifications. All queries are read-only and can be parallelized.

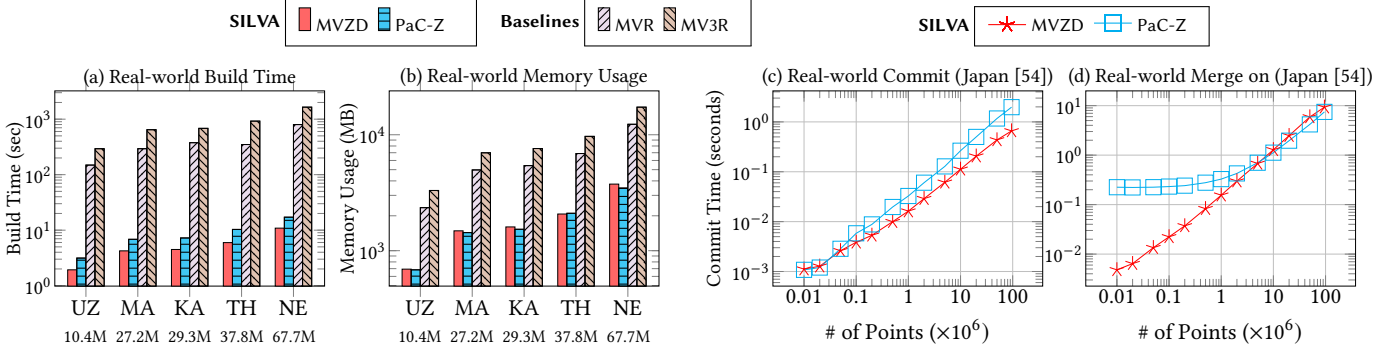


Figure 12: Real-world OSM data evaluation (in different regions), lower is better.

introducing extra overhead. On the other hand, SILVA provides fundamental version-aware interfaces such as spatial diff, commit, and merge. These interfaces are crucial building blocks in multi-version scenarios, such as the agentic AI systems. We implemented three indexes in SILVA and conducted extensive experiments to evaluate their effectiveness. According to the experimental results, SILVA consistently achieves the best or comparable to the best performance with significant improvement in memory footprint.

References

- [1] Sattam Alsubaiee, Yasser Altowim, Hotham Altwaijry, Alexander Behm, Vinayak R. Borkar, Yingyi Bu, Michael J. Carey, Inci Cetindil, Madhusudan Cheelang, Khurram Faraaz, Eugenia Gabrielova, Raman Grover, Zachary Heilbron, Young-Seok Kim, Chen Li, Guangqiang Li, Ji Mahn Ok, Nicola Onose, Pouria Pirzadeh, Vassilis J. Tsotras, Rares Vernica, Jian Wen, and Till Westmann. 2014. AsterixDB: A Scalable, Open Source BDMS. *Proceedings of the VLDB Endowment (PVLDB)* (2014).
- [2] J Chris Anderson, Jan Lehnardt, and Noah Slater. 2010. *CouchDB: the definitive guide: time to relax.* O'Reilly Media, Inc.
- [3] Lars Arge, Klaus H Hinrichs, Jan Vahrenhold, and Jeffrey Scott Vitter. 2002. Efficient bulk operations on dynamic R-trees. *Algorithmica* 33 (2002), 104–128.
- [4] Nimar S Arora, Robert D Blumofe, and C Greg Plaxton. 2001. Thread scheduling for multiprogrammed multiprocessors. *Theory of Computing Systems (TOCS)* 34, 2 (2001), 115–144.
- [5] Bruno Becker, Stephan Gschwind, Thomas Ohler, Bernhard Seeger, and Peter Widmayer. 1996. An Asymptotically Optimal Multiversion B-Tree. *The VLDB Journal* 5, 4 (1996), 264–275.
- [6] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-tree: An efficient and robust access method for points and rectangles. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 322–331.
- [7] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (1975), 509–517.
- [8] Jon Louis Bentley. 1977. *Algorithms for Klee's rectangle problems*. Technical Report. Carnegie-mellon University.
- [9] Philip A Bernstein, Colin W Reid, and Sudipto Das. 2011. Hyder-A Transactional Record Manager for Shared Flash.. In *Conference on Innovative Data Systems Research (CIDR)*, Vol. 11. 9–20.
- [10] Anant Bhardwaj, Amol Deshpande, Aaron J. Elmore, David Karger, Samuel Madden, Aditya G. Parameswaran, Harihar Subramanyam, Eugene Wu, and Rebecca Zhang. 2015. Collaborative data analytics with DataHub. *Proc. VLDB Endow.* 8, 12 (2015), 1916–1919.
- [11] Guy Blelloch, Daniel Ferizovic, and Yihan Sun. 2022. Joinable Parallel Balanced Binary Trees. *ACM Transactions on Parallel Computing (TOPC)* 9, 2 (2022), 1–41.
- [12] Guy E. Blelloch, Daniel Anderson, and Laxman Dhulipala. 2020. ParlayLib — a toolkit for parallel algorithms on shared-memory multicore machines. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 507–509.
- [13] Guy E Blelloch and Magdalen Dobson. 2022. Parallel Nearest Neighbors in Low Dimensions with Batch Updates. In *Algorithm Engineering and Experiments (ALENEX)*. SIAM, 195–208.
- [14] Guy E. Blelloch, Daniel Ferizovic, and Yihan Sun. 2016. Just Join for Parallel Ordered Sets. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.
- [15] Guy E. Blelloch, Jeremy T. Fineman, Yan Gu, and Yihan Sun. 2020. Optimal parallel algorithms in the binary-forking model. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 89–102.
- [16] Robert D. Blumofe and Charles E. Leiserson. 1993. Space-efficient scheduling of multithreaded computations. In *ACM Symposium on Theory of Computing (STOC)*. ACM, 362–371.
- [17] Boost geometry 2024. Boost Geometry Library. <https://www.boost.org/library/latest/geometry/>.
- [18] Surajit Chaudhuri and Vivek R. Narasayya. 1998. AutoAdmin 'What-if' Index Analysis Utility. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 367–378.
- [19] Jack Chen, Samir Jindel, Robert Walzer, Rajkumar Sen, Nika Jimshelishvili, and Michael Andrews. 2016. The MemSQL Query Optimizer: A modern optimizer for real-time analytics in a distributed database. *Proceedings of the VLDB Endowment (PVLDB)* 9, 13 (2016), 1401–1412. doi:10.14778/3007263.3007277
- [20] Chicago Data Portal 2024. Chicago Crimes: 2001 to Present. https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2/about_data.
- [21] Demographic and Economic Data 2022. TIGER with Selected Demographic and Economic Data. <https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-data.html>.
- [22] Laxman Dhulipala, Guy E. Blelloch, Yan Gu, and Yihan Sun. 2022. PaC-trees:

- Supporting Parallel and Compressed Purely-Functional Collections. In *ACM Conference on Programming Language Design and Implementation (PLDI)*.
- [23] Laxman Dhulipala, Guy E Blelloch, and Julian Shun. 2019. Low-latency graph streaming using compressed purely-functional trees. In *ACM Conference on Programming Language Design and Implementation (PLDI)*. 918–934.
 - [24] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Ake Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. Hekaton: SQL server's memory-optimized OLTP engine. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1243–1254.
 - [25] Ahmed Eldawy, Vagelis Hristidis, Saheli Ghosh, Majid Saeedan, Akil Sevim, A. B. Siddique, Samridhi Singla, Ganesh Sivaram, Tin Vu, and Yaming Zhang. 2021. Beast: Scalable Exploratory Analytics on Spatio-temporal Data. In *ACM International Conference on Information and Knowledge Management*. 3796–3807.
 - [26] Ahmed Eldawy and Mohamed F. Mokbel. 2015. SpatialHadoop: A MapReduce Framework for Spatial Data. In *IEEE International Conference on Data Engineering (ICDE)*. 1352–1363.
 - [27] Raphael A Finkel and Jon Louis Bentley. 1974. Quad trees a data structure for retrieval on composite keys. *Acta informatica* 4 (1974), 1–9.
 - [28] Peter Frühwirth, Marcus Huber, Martin Mulazzani, and Edgar R Weippl. 2010. InnoDB database forensics. In *2010 24th IEEE International Conference on Advanced Information Networking and Applications*. IEEE, 1028–1036.
 - [29] Volker Gaede and Oliver Günther. 1998. Multidimensional Access Methods. *ACM Comput. Surv.* 30, 2 (1998), 170–231.
 - [30] Junhao Gan and Yufei Tao. 2017. On the hardness and approximation of Euclidean DBSCAN. *ACM Transactions on Database Systems (TODS)* 42, 3 (2017), 1–45.
 - [31] geogig 2020. GeoGig: Distributed Versioned Spatial Data Editing. <https://geogig.org/>.
 - [32] Yan Gu, Zachary Napier, and Yihan Sun. 2022. Analysis of Work-Stealing and Parallel Cache Complexity. In *SIAM Symposium on Algorithmic Principles of Computer Systems (APOCS)*. SIAM, 46–60.
 - [33] Antonin Guttman. 1984. R-trees: A dynamic index structure for spatial searching. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 47–57.
 - [34] Silu Huang, Liqi Xu, Jialin Liu, Aaron J. Elmore, and Aditya G. Parameswaran. 2017. OrpheusDB: Bolt-on Versioning for Relational Databases. *Proc. VLDB Endow.* 10, 10 (2017), 1130–1141.
 - [35] Edwin H. Jacox and Hanan Samet. 2007. Spatial join techniques. *ACM Trans. Database Syst.* (2007).
 - [36] Ibrahim Kamel and Christos Faloutsos. 1992. Parallel R-trees. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 21, 2 (1992), 195–204.
 - [37] kart 2024. Kart: Distributed Version Control for Geospatial Data. <https://kartproject.org/>.
 - [38] Alfons Kemper and Thomas Neumann. 2011. HyPer: A hybrid OLTP&OLAP main memory database system based on virtual memory snapshots. In *IEEE International Conference on Data Engineering (ICDE)*, Serge Abiteboul, Klemens Böhm, Christoph Koch, and Kian-Lee Tan (Eds.). IEEE Computer Society, 195–206.
 - [39] Curtis P. Kolovson and Michael Stonebraker. 1991. Segment Indexes: Dynamic Indexing Techniques for Multi-Dimensional Interval Data. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM Press, 138–147.
 - [40] Per-Ake Larson, Spyros Blanas, Cristian Diaconu, Craig Freedman, Jignesh M Patel, and Mike Zwilling. 2011. High-performance concurrency control mechanisms for main-memory databases. *Proceedings of the VLDB Endowment (PVLDB)* 5, 4 (2011), 298–309.
 - [41] Scott T. Leutenegger, Jeffrey Edgington, and Mario Alberto López. 1997. STR: A Simple and Efficient Algorithm for R-Tree Packing. In *IEEE International Conference on Data Engineering (ICDE)*. 497–506.
 - [42] Justin J. Levandoski, David B. Lomet, and Sudipta Sengupta. 2013. The Bw-Tree: A B-tree for New Hardware Platforms. In *IEEE International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 302–313.
 - [43] libspatialindex 2024. libspatialindex. <https://libspatialindex.org/en/latest/>.
 - [44] lightning 2015. Lightning memory-mapped database manager (LMDB). <https://lMDB.io/db/lmdb>.
 - [45] Los Angeles Open Data 2024. Traffic Collision Data from 2010 to Present. https://data.lacity.org/Public-Safety/Traffic-Collision-Data-from-2010-to-Present/d5tf-ez2w/about_data.
 - [46] Michael Maddox, David Goehring, Aaron J. Elmore, Samuel Madden, Aditya G. Parameswaran, and Amol Deshpande. 2016. Decibel: the relational dataset branching system. *Proc. VLDB Endow.* 9, 9 (2016), 624–635.
 - [47] Ziyang Men, Bo Huang, Yan Gu, and Yihan Sun. 2026. Parallel Dynamic Spatial Indexes. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 150–163.
 - [48] Ziyang Men, Zheqi Shen, Yan Gu, and Yihan Sun. 2025. Parallel kd-tree with Batch Updates. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 3, 1 (2025), 62:1–62:26.
 - [49] MicrosoftSQL 2025. Microsoft SQL server. <https://www.microsoft.com/en-us/sql-server/>.
 - [50] mongodb 2025. MongoDB. <https://www.mongodb.com/>.
 - [51] Guy M Morton. 1966. A computer oriented geodetic data base and a new technique in file sequencing. (1966).
 - [52] Mario A. Nascimento and Jefferson R. O. Silva. 1998. Towards Historical R-trees. In *ACM symposium on Applied Computing (SAC)*. ACM, 235–240.
 - [53] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta Inf.* 33, 4 (1996).
 - [54] Open Street Map Data 2024. Source of OSM data. <https://download.geofabrik.de/>.
 - [55] oracle 2025. Oracle Database. <https://www.oracle.com/database/>.
 - [56] Jack A. Orenstein. 1986. Spatial Query Processing in an Object-Oriented Database System. In *SIGMOD Conference*. ACM Press, 326–336.
 - [57] Jack A. Orenstein and T. H. Merrett. 1984. A Class of Data Structures for Associative Searching. In *PODS*. ACM, 181–190.
 - [58] PostGIS 2025. PostGIS. <https://postgis.net/>.
 - [59] Frank Ramsak, Volker Markl, Robert Fenk, Martin Zirkel, Klaus Elhardt, and Rudolf Bayer. 2000. Integrating the UB-Tree into a Database System Kernel. In *VLDB*. Morgan Kaufmann, 263–272.
 - [60] Redis 2024. Redis: The Real-Time Data Platform. <https://redis.io/>.
 - [61] Simonas Saltenis, Christian S. Jensen, Scott T. Leutenegger, and Mario Alberto López. 2000. Indexing the Positions of Continuously Moving Objects. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, 331–342.
 - [62] SILVA Library 2025. Implementation of SILVA: Spatial Index Library with Versioned Access. <https://github.com/ItakEjgo/mvzd>.
 - [63] spatialhadoop 2025. SpatialHadoop. <https://spatialhadoop.cs.umn.edu/>.
 - [64] Michael Stonebraker, Samuel Madden, Daniel J. Abadi, Stavros Harizopoulos, Nabil Hachem, and Pat Helland. 2007. The End of an Architectural Era (It's Time for a Complete Rewrite). In *Proceedings of the VLDB Endowment (PVLDB)*. ACM, 1150–1160.
 - [65] Yihan Sun and Guy Blelloch. 2019. Implementing Parallel and Concurrent Tree Structures. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 447–450.
 - [66] Yihan Sun, Guy E Blelloch, Wan Shen Lim, and Andrew Pavlo. 2019. On supporting efficient snapshot isolation for hybrid workloads with multi-versioned indexes. *Proceedings of the VLDB Endowment (PVLDB)* 13, 2 (2019), 211–225.
 - [67] Yihan Sun, Daniel Ferizovic, and Guy E Blelloch. 2018. PAM: Parallel Augmented Maps. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.
 - [68] Yufei Tao and Dimitris Papadias. 2001. MV3R-Tree: A Spatio-Temporal Access Method for Timestamp and Interval Queries. In *Proceedings of the VLDB Endowment (PVLDB)*. Morgan Kaufmann, 431–440.
 - [69] Yufei Tao, Dimitris Papadias, and Jimeng Sun. 2003. The TPR*-Tree: An Optimized Spatio-Temporal Access Method for Predictive Queries. In *Proceedings of the VLDB Endowment (PVLDB)*. 790–801.
 - [70] John Woodfill and Michael Stonebraker. 1983. An Implementation of Hypothetical Relations. In *Proceedings of the VLDB Endowment (PVLDB)*. Morgan Kaufmann, 157–166.
 - [71] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An Empirical Evaluation of In-Memory Multi-Version Concurrency Control. *Proceedings of the VLDB Endowment (PVLDB)* 10 (2017), 781–792. Issue 7.

A Algorithm Details for PaC-Z tree

PaC-Z tree reuses parallel algorithms for tree construction and update. For page limit, we refer readers to [22] for the details of update algorithms, and focus here on PaC-Z tree's spatial query algorithms proposed by this paper. Starting with queries applicable to any single version, we apply standard spatial pruning via bounding boxes. We can use bounding boxes to determine whether a subtree is irrelevant to a spatial query—e.g., when its box is out of the query range in a range report query, or the box is further than the current k -th nearest candidate in a k -nn query—and skip the entire subtree if so. Range count queries can be further accelerated when the subtree is entirely within the query range, and the cardinality of the subtree can be directly added to the result.

Beyond the standard spatial queries applicable to a single version, we also introduce version-aware query algorithms, which offer more sophisticated querying capabilities. For spatial-diff queries, the high-level idea is to traverse two trees T_1 and T_2 simultaneously, and detect the unchanged sub-regions by comparing tree nodes. If the two trees shared the same subtree root, then there are no changes to report (i.e., $T_1 = T_2$). Otherwise, we first use the split function in PaC-tree, which splits one tree (typically the larger one) by the root key r of the other tree. Without loss of generality, assume we split T_2 by T_1 . Assume T_1 has left (right) subtree L_1 (R_1) with root key k_1 , then we will use the split function to split T_2 , obtaining two subtrees: $T_2^{<k_1}$ and $T_2^{>k_1}$, which contains all entries in T_2 that are less/greater than k_1 , respectively. Then, we recursively process the subtree pairs: $\langle L_1, T_2^{<k_1} \rangle$ and $\langle R_1, T_2^{>k_1} \rangle$ to collect spatial diff results. This approach is similar to the one in our spatial-invariance aware index MVZD-tree, described in Sec. 5.

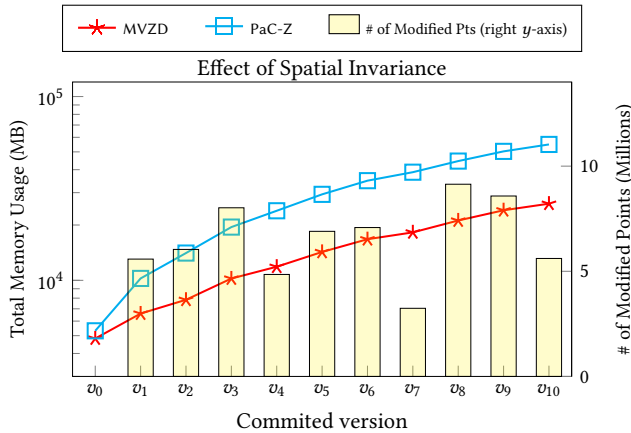


Figure 13: Effect of Spatial Invariance. The line chart (left y-axis) shows total memory usage storing all versions, while the bars (right y-axis) represent the count of modified points in millions for each new version. v_0 is the initial version with 100M points. **Bo: move to technical report.**

B Batch Deletion Algorithm For MVZD-Tree

Alg. 5 shows the procedure of BATCHDELETE, which follows a similar approach to batch insertion, with some differences for the leaf case and handling underflow after deletion. Similar to BATCHINSERT, the entry function BATCHDELETE sorts the point set D and initiates the recursive call to BATCHDELETENODE. BATCHDELETENODE copies

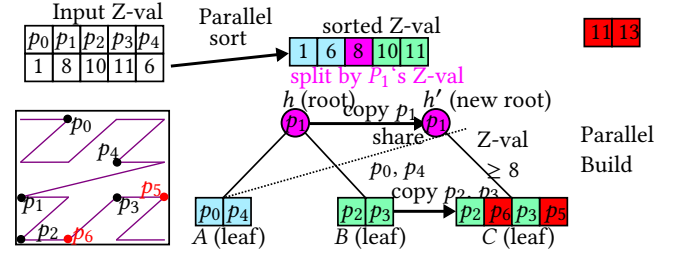


Figure 14: PaC-Z tree Example.

Algorithm 5: BATCHDELETE

Input: A version handle h , a point set D to be deleted.
Output: New version handle after deletion
Parameter: c : leaf capacity.
 b^* : maximum Z-value binary representation length.

```

1 Function BATCHDELETE( $h, D$ ) // BATCHDELETE entry
2   if  $D = \emptyset$  then return  $h$ 
3   calculate Z-values for points in  $D$  and sort them by Z-values
4   return BATCHDELETENODE( $h, D, 0, |D|, b^*$ )
5 Function BATCHDELETENODE( $h, D, l, r, b$ )
6    $x \leftarrow$  new tree node by copying  $h$ 
7   if  $x$  is leaf then
8     delete  $\{D_l, \dots, D_r\}$  from  $x.points$  // achieved by linear scan
9     if  $x.points = \emptyset$  then free node  $x$ 
10    return  $x$ 
11    $mid \leftarrow$  the minimum  $i \in [l, r]$  with the  $b$ -th bit in  $D_i$  as 1
12   In Parallel:
13     if  $l < mid$  then
14        $x.lc \leftarrow$  BATCHDELETENODE( $h.lc, D, l, mid, b-1$ )
15     if  $mid < r$  then
16        $x.rc \leftarrow$  BATCHDELETENODE( $h.rc, D, mid, r, b-1$ )
17   if  $lc = \text{nullptr}$  and  $rc = \text{nullptr}$  then free node  $x$ 
18   else if  $x.lc = \text{nullptr}$  then  $x \leftarrow x.rc$  // lift the right child
19   else if  $x.rc = \text{nullptr}$  then  $x \leftarrow x.lc$  // lift the left child
20   else if  $|x.lc.points| + |x.rc.points| \leq c$  then // form a new leaf
21      $x \leftarrow$  new leaf node stores points in  $x.lc.points \cup x.rc.points$ 
22   update augmented values for  $x$ 
23   return  $x$ 

```

the given handle and returns the copied handle after deletion. If we touch a leaf node, we delete points from the existing stored points in line 8. Note that since both stored points and deleted points are sorted, we can achieve this by linear scanning. If the tree node becomes empty, it is freed on line 9. Notice that removing points from a leaf node might cause an underflow to its parent node which is checked by the calling function as shown below. If h points to an interior node, lines 11–16 split the point list and recursively delete data from both subtrees. Underflow handling starts from line 17 after the recursive calls. If both child pointers are nullptr, h is freed and replaced with nullptr in line 17. When only one child pointer exists, it is lifted accordingly in line 18 or line 19. Otherwise, we check whether two child nodes can be merged into a leaf node in line 21. Finally, we update the augmented values of the interior node in line 22.

The underflow handling keeps MVZD-tree compact, reduces tree depth, and speeds up spatial queries.

C Theoretical Analysis

This section presents the work and span analysis for version operations in SILVA: BUILD, BATCHINSERT, BATCHDELETE, and COMMIT. We omitted spatial queries and merge since they are highly data

sensitive and the analysis on the worst case becomes trivial. Since all three indexes use *leaf wrapping* technique, we use B (usually 32) to represent the leaf node capacity and assume it is a constant.

C.1 PaC-Z Tree

THEOREM C.1 (BUILD COST). *Given a set of points P with size n , PaC-Z tree can be built in $O(n \log n)$ work with $O(\log n)$ span.*

Proof. The build procedure first compute Z-values for all points in $O(n)$ work with $O(\log n)$ span, then sort them in $O(n \log n)$ work with $O(\log n)$ span (using optimal parallel comparison-based sorting), and finally build a perfect balanced binary search tree use divide-and-conquer manner in $O(n)$ work with $O(\log n)$ span. Therefore, build work is $O(n + n \log n + n) = O(n \log n)$, and build span is $O(\log n + \log n + \log n) = O(\log n)$. $\square$

LEMMA C.2 (BATCH INSERT/DELETE COST). *Given a PaC-Z tree T built upon point set P with size n , and a point set P' with size m . Batch insert/delete P' to/from T can be processed in $O(m \log n)$ work with $O(\log m \log n)$ span.*

Proof. We first analyze batch insertion first, and the analysis of batch delete is similar. The batch insert procedure contains three parts: 1) partition P' into subtrees until meet the base cases; 2) process base cases (append inserted points to existing leaf; dealing with leaf overflow); and 3) rebalance after insertion. Firstly, node copy for a tree node has $O(1)$ cost. Since both interior and leaf nodes have constant number of fields. For the partition part, T has $O(\log n)$ levels, and in each level the partition procedure cost $O(m)$ work with $O(\log m)$ span. Therefore, the partition work in total is $O(m \log n)$ and the partition span is $O(\log m \log n)$. For the base case, we need to touch all updated points in the worst case. Therefore the work is $O(B + m) = O(m)$ and span is $O(\log(B + m)) = O(\log m)$. For the rebalance part, PaC-Z tree uses concat primitive [67] for rebalancing. Each concat can combine two trees with size x and y in $O(\log \frac{x}{y})$ work and span ($x > y$). There are $O(m)$ leaf nodes need to be rebalanced, and thus $O(m)$ concat will be called. Each concat costs at most $O(\log \frac{n}{m})$ work and span (w.l.o.g. assume $n > m$). Therefore the total work for rebalance is $O(m \log \frac{n}{m})$, and the span is $O(\log \frac{n}{m})$.

Adding up the three parts, the insertion work is $O(m \log n + m + m \log \frac{n}{m}) = O(m \log n)$, and insertion span is $O(\log m \log n + \log m + \log \frac{n}{m}) = O(\log m \log n)$, bounded by partition. $\square$

THEOREM C.3 (COMMIT COST). *Given a PaC-Z tree T build upon point set P with size n , delete point set D with size m_1 , and insert point set I with size m_2 . A new version T' can be committed in $O((m_1 + m_2) \log n)$ work with $O(\log(m_1 m_2) \log n)$ span.*

Proof. Commit consists of three steps. Batch deletion, batch insertion, and purge the intermediate version. First we use Lem. C.2 to get the costs for batch insertion and batch deletion. For batch deletion, we create a temporary version T_t by deleting D from T , which costs $O(m_1 \log n)$ work and $O(\log m_1 \log n)$ span. Then we batch insert I into T_t . Since m_1 points are deleted from P , the size of T_t is $n - m_1$. Therefore the batch insertion costs work $O(m_2 \log(n - m_1))$ and span $O(\log m_2 \log(n - m_1))$. Finally, we purge T_t , which contains $n - m_1$ points. The work and span for purge are the same as traverse all tree nodes in parallel, i.e., $O(n - m_1)$ work and $O(\log(n - m_1))$ span.

Combine all three parts together, we obtain that the commit procedure costs with $O(m_1 \log n + m_2 \log(n - m_1) + (n - m_1)) = O((m_1 + m_2) \log n)$ work and $O(\log m_1 \log n + \log m_2 \log(n - m_1)) = O(\log(m_1 m_2) \log n)$ span. $\square$

C.2 MVZD-Trees

Given a spatial dataset P with n points, let $d_{\max}$ ($d_{\min}$) be the furthest (closest) pairwise distance in P , respectively. The aspect ratio of P , denoted by Δ_P , is defined to be $\Delta_P = d_{\max}/d_{\min}$.

THEOREM C.4 (BUILD COST). *Given a set P with n points, a MVZD-tree can be built on P in $O(n \log n)$ work with $O(\log n \log \Delta_P)$ span.*

Proof. The first step in the build algorithm is to compute and sort by Z-values which requires $O(n \log n)$ work and $O(\log n)$ span when using optimal parallel comparison-based sorting.

To compute the cost of the tree construction step, we observe that MVZD-tree is a binary tree with $O(n)$ leaf nodes (each leaf contains at least 1 point), and the total number of tree nodes is $O(n)$. For each tree node, a binary search with cost $O(\log n)$ is performed to partition the points (see line 10). Therefore, the total work is $O(n \log n)$. For the span, the height of MVZD-tree is determined by the aspect ratio Δ_P . Since the space is halved for each level, the height of MVZD-tree is $O(\log \Delta_P)$. For each level of recursion, a binary search is performed to partition the points into two parts, therefore the span is $O(\log \Delta_P \log n)$. $\square$

LEMMA C.5 (BATCH INSERT/DELETE COST). *Given a MVZD-tree T maintaining dataset P with size n , and a point set P' with size m . Batch inserted/deleted P' to/from T can be processed with $O(m \log \Delta_P)$ work and $O(\log \Delta_P \log m)$ span.*

Proof. We show the proof of batch insertion here, and the proof for deletion is similar. Firstly, the tree after inserting P' into P has the same shape as directly constructing a MVZD-tree on top of $P \cup P'$ since MVZD-tree is partitioned using spatial median — the shape is determined once the global region is provided. Given the fixed node size, the node copy procedure costs $O(1)$ operations. Therefore the tree height after batch insertion is $O(\log \Delta_{P \cup P'})$.

We analyze the work first. The tree height of T is $O(\log \Delta_P)$. For each level of the tree, we need to partition entire P' once. The partition procedure has $O(m)$ work and $O(\log m)$ span. Therefore, the total work of batch insertion is $O(m \log \Delta_P)$.

For the span part, consider the longest path during the insertion. It touches $O(\log \Delta_P)$ tree nodes (a chain from root to leaf), and for each touch it requires a $O(\log m)$ binary search to partition (as shown in line 15). Therefore, the span is $O(\log \Delta_P \log m)$. $\square$

THEOREM C.6 (COMMIT COST). *Given a MVZD-tree T built for point set P with size n , delete point set D with size m_1 , and insert point set I with size m_2 . A new version T' can be committed in $O((m_1 + m_2) \log \Delta_P)$ work with $O(\log m_1 m_2 \log \Delta_P)$ span.*

Proof. We consider the three steps for commit: 1) Batch delete D ; 2) Batch insert I ; and 3) Purge the intermediate version.

For the first step, we create a temporary version T_t by batch deleting D , which costs $O(m_1 \log \Delta_P)$ work with $O(\log \Delta_P \log m_1)$ span according to Lem. C.5. T_t is the same shape as directly built on point set $P \setminus D$. Therefore, the second step costs $O(m_2 \log \Delta_{P \setminus D})$ work and $O(\log \Delta_{P \setminus D} \log m_2)$ span. To purge T_t , we just need to traverse T_t in parallel with $O(n - m_1)$ work and $O(\log \Delta_{P \setminus D})$ span.

Since $|P \setminus D| < |P|$, then $O(\log \Delta_{P \setminus D} \log m_2) = O(\log \Delta_P \log m_2)$.

By combining the three parts, the commit costs $O(m_1 \log \Delta_P + m_2 \log \Delta_P + n - m_1) = O((m_1 + m_2) \log \Delta_P)$ work and $O(\log \Delta_P \log m_1 + \log \Delta_P \log m_2 + \log \Delta_P) = O(\log m_1 m_2 \log \Delta_P)$ span. $\square$

D TODO

- (1) call MVQ-tree.
- (2) add a technical report.
- (3) change pseudocode leaf capacity $c \rightarrow B$, consistent with the description.
- (4) move proof to appendix.
- (5) add figure split information, its fine to make them two columns.
- (6) move ZD-DT to baselines section.
- (7) mention query performance in case study.
- (8) add a section discuss memory comparison of MVQ-tree and PaC-Z tree.